

Big Data - Proyecto Parcial 2026-2

Accilio Villanueva, Ana María
Cabezas Ramírez, Dylan Andrés
Gálvez Pacori, José Guillermo
Meneses Roncal, Matías Alonso
Salinas Salas, Joaquín Mauricio

Septiembre 2026

Índice

1. Introducción	2
2. Dataset	2
2.1. Origen	2
2.2. Estructura	2
2.3. Hallazgos de la exploración	3
2.4. Métricas de negocio (vista preliminar)	3
3. Implementación en GCP	4
3.1. Arquitectura general	4
3.2. Diseño de Arquitectura Big Data	4
3.3. Almacenamiento: bucket de Cloud Storage	6
3.4. Clúster Dataproc	8
3.5. Reproducibilidad con Terraform	9
4. Procesamiento distribuido	9
4.1. Metodología	9
4.2. Polars	10
4.3. Apache Spark	10
4.4. Modin	11
4.5. Dask	11
4.6. Resultados y comparación	11
4.7. Comparación de rendimiento	12
5. Operaciones	14
5.1. Operaciones tipo CRUD sobre el dataset	14
5.2. Validación y limpieza	14
5.3. Agrupaciones y agregaciones	15
5.4. Métricas de negocio	16
6. Hadoop MapReduce	16
6.1. Preparación de los inputs	16
6.2. Los tres programas	17
6.3. Comando ejecutado	17
6.4. Evidencia y resultados	18
7. Anexo: Repositorio del proyecto	18

1. Introducción

El presente informe documenta el desarrollo del Proyecto Grupal I del curso de Big Data, cuyo objetivo es aplicar de forma práctica los conceptos de almacenamiento distribuido, procesamiento paralelo y programación en la nube sobre un conjunto de datos real.

Para ello, el grupo trabajó con un dataset de transacciones de comercio electrónico (*Online Retail II*) que supera el millón de registros, y construyó una arquitectura completa sobre Google Cloud Platform (GCP): un data lake en Cloud Storage, un clúster de Google Cloud Dataproc, y un pipeline de procesamiento implementado en cuatro frameworks distintos (Polars, Dask, Modin y Apache Spark), además de tres programas de Hadoop MapReduce ejecutados sobre HDFS.

El objetivo no fue únicamente demostrar que cada herramienta “corre”, sino resolver, con las cuatro, el mismo conjunto de doce consultas de negocio sobre los datos, limpieza, deduplicación, transformación, filtrado, agregaciones, agrupaciones, ordenamiento y cálculo de métricas, para poder comparar de forma directa cómo cada framework resuelve exactamente el mismo problema. Toda la infraestructura de GCP (bucket y clúster) se define como código mediante Terraform, lo que permite recrearla o destruirla de forma reproducible.

Las secciones siguientes describen, en orden: el dataset elegido y su calidad de datos, la implementación del almacenamiento y la arquitectura sobre GCP, el procesamiento distribuido con los cuatro frameworks, las operaciones concretas realizadas sobre los datos, y finalmente la ejecución de Hadoop MapReduce sobre el clúster.

2. Dataset

2.1. Origen

El dataset utilizado es **Online Retail II**, publicado en el repositorio UCI Machine Learning Repository (donado en 2019 por Daqing Chen, DOI 10.24432/C5CG6D). Contiene transacciones reales de venta de un minorista en línea del Reino Unido especializado en artículos de regalo únicos, comprados en su mayoría por otros negocios (mayoristas). Cubre el periodo del 1 de diciembre de 2009 al 9 de diciembre de 2011, es decir, dos años fiscales completos.

El archivo original se distribuye en formato Excel con dos hojas (**Year 2009-2010** y **Year 2010-2011**), para este proyecto se utiliza la versión combinada en un único archivo CSV (**data/online_retail_II.csv**).

Se eligió este dataset por tres razones concretas: tiene contexto real de negocio (retail / e-commerce), su volumen supera cómodamente el mínimo de 500,000 filas pedido por el enunciado, y, a diferencia de un dataset ya limpio, presenta problemas de calidad de datos genuinos, lo que da material real para las tareas de limpieza, deduplicación y transformación que exige la Sección 4 del proyecto.

2.2. Estructura

- **Formato:** CSV tabular, un registro por línea de producto dentro de una factura.
- **Filas:** 1,067,371 registros.
- **Columnas:** 8.

Variable	Tipo	Descripción
Invoice	texto	Número de factura. Prefijo C indica cancelación/devolución.
StockCode	texto	Código identificador del producto.
Description	texto	Nombre/descripción del producto.
Quantity	entero	Cantidad de unidades. Puede ser negativa (devoluciones).
InvoiceDate	fecha-hora	Fecha y hora de la transacción.
Price	decimal	Precio unitario (libras esterlinas, GBP).
Customer ID	numérico	Identificador del cliente. Puede ser nulo.
Country	texto	País donde se registró la transacción.

Cuadro 1: Variables principales del dataset

2.3. Hallazgos de la exploración

Antes de implementar cualquier consulta, se realizó una exploración completa del dataset (ver el notebook `notebooks/exploracion.ipynb`) para entender qué tan “sucio” es en la práctica:

Hallazgo	Cantidad	%
Filas totales	1,067,371	100 %
Customer ID nulo	243,007	22.77 %
Description nula	4,382	0.41 %
Filas duplicadas exactas	34,335	3.22 %
Facturas canceladas (prefijo C)	19,494	—
Quantity negativa	22,950	2.15 %
Price ≤ 0	6,207	0.58 %
Países distintos	43	—
Reino Unido (% del total)	981,330	91.94 %
Facturas únicas (Invoice)	53,628	—
Clientes únicos (Customer ID)	5,942	—
Productos únicos (StockCode)	5,305	—

Cuadro 2: Hallazgos de calidad de datos sobre el dataset completo

El rango temporal cubierto va de 2009-12-01 a 2011-12-09.

Estos hallazgos justifican directamente la Sección 4 del proyecto: casi un cuarto de las filas no tiene cliente asociado, un 3.22 % del dataset son filas duplicadas exactas, y hay filas con precio negativo o cero que no representan ventas reales. Todo esto se trata explícitamente en las 12 consultas implementadas por cada framework (ver Sección 4).

Los 34,335 duplicados de la tabla anterior se calculan sobre el archivo crudo, antes de cualquier limpieza. En el pipeline de las 12 consultas, la deduplicación (consulta 2) corre después de la limpieza inicial (consulta 1), que ya descarta las filas con `Description` nula o `Price` ≤ 0 . Como el conjunto sobre el que se cuentan los duplicados ya es distinto, el número de duplicados eliminados en esa etapa del pipeline es 34,147 y no 34,335 (ver Sección 5), ambas cifras son correctas, cada una sobre un conjunto de datos distinto.

2.4. Métricas de negocio (vista preliminar)

Sobre el subconjunto de transacciones con `Quantity` positiva (ventas efectivas, sin cancelaciones):

- **Revenue total:** £20,814,291.998
- **Ticket promedio por factura:** £359.65
- **Producto más vendido por unidades:** WORLD WAR 2 GLIDERS ASSTD DESIGNS (108,545 unidades)
- **Cliente con mayor gasto acumulado:** 18102.0 (£598,215.22)

3. Implementación en GCP

3.1. Arquitectura general

La arquitectura del proyecto sigue el flujo estándar de un pipeline de Big Data: primero la fuente de datos externa, luego la ingesta batch, después el almacenamiento (data lake) y finalmente el procesamiento distribuido, todo dentro de Google Cloud Platform. El dataset se carga una única vez a Cloud Storage (es un histórico cerrado, no un flujo continuo de eventos, por lo que la ingesta es batch y no streaming), y desde ahí lo consumen tanto los cuatro frameworks de procesamiento (Sección 4) como el pipeline de Hadoop MapReduce (Sección 6).

Toda la infraestructura, el bucket y el clúster, se define como código mediante Terraform (`terraform/main.tf`), lo que permite recrearla o destruirla de forma reproducible con un solo comando, sin pasos manuales en la consola.

3.2. Diseño de Arquitectura Big Data

La arquitectura definida para el proyecto está basada por completo en GCP, e incluye los cuatro componentes pedidos: fuente de datos (*Online Retail II*), ingesta batch (carga única del dataset a Cloud Storage), almacenamiento (data lake en tres zonas dentro de un bucket de Cloud Storage) y procesamiento distribuido (clúster Dataproc, detallado en la Sección 4).

Diagrama de arquitectura

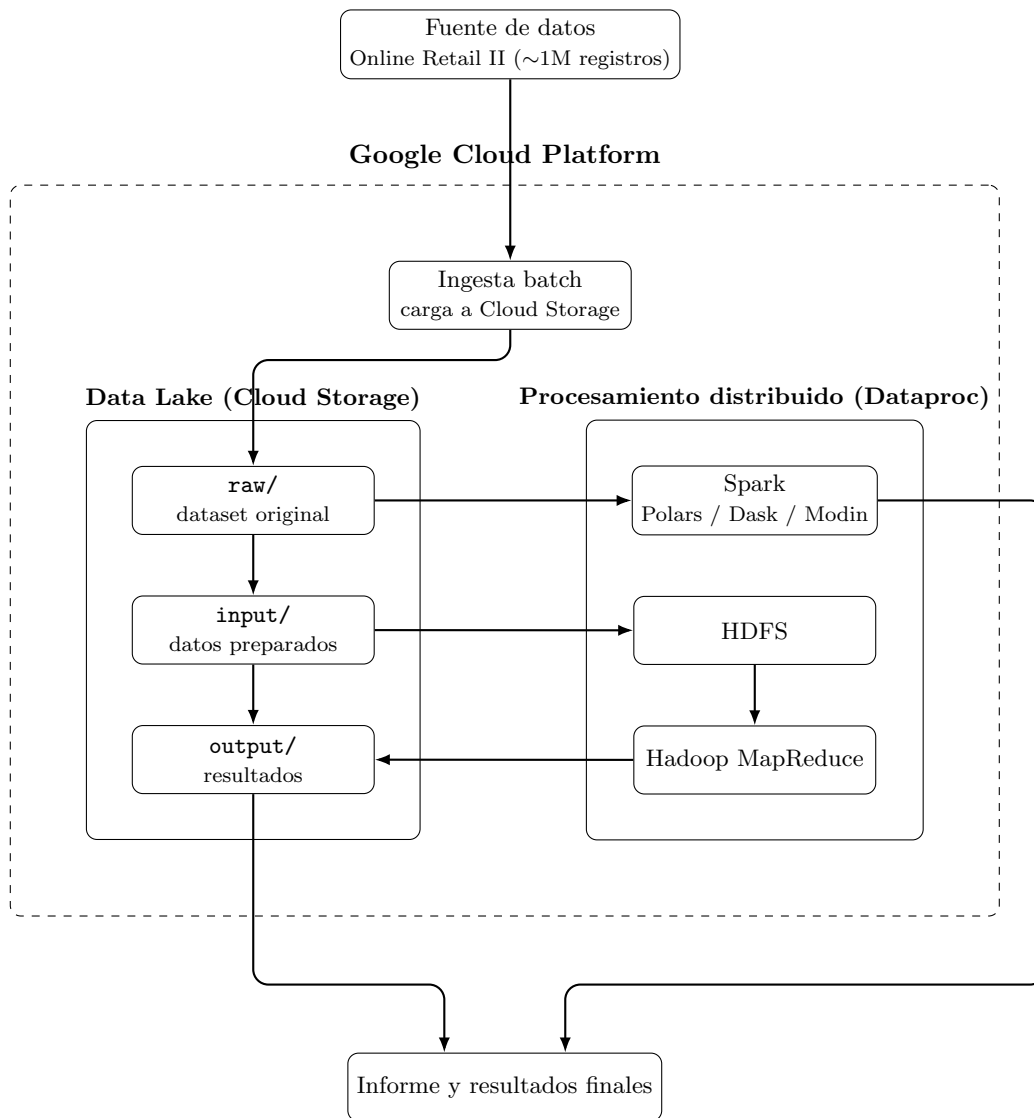


Figura 1: Diagrama de arquitectura general del proyecto

Explicación de componentes

Componente	Qué es	Por qué está ahí
Fuente de datos	<i>Online Retail II</i> , transacciones de e-commerce retail (Reino Unido), ~1.06M filas	Dataset con contexto real de negocio y volumen suficiente para justificar procesamiento distribuido
Ingesta (batch)	Carga del dataset completo a Cloud Storage	Es batch, no streaming, porque el dataset es un histórico fijo, no un flujo continuo de eventos
Almacenamiento (Data Lake)	Un bucket de Cloud Storage organizado en 3 zonas (<i>raw/</i> , <i>input/</i> , <i>output/</i>)	Cloud Storage separa cómputo de almacenamiento, las 3 zonas simulan las etapas típicas de un data lake: cruda, preparada, y resultados
Procesamiento distribuido	Clúster Dataproc (1 master + 2 workers) corriendo Spark, Polars/Dask/Modin, y HDFS + Hadoop MapReduce	Dataproc administra Spark, Hadoop y HDFS de forma nativa sobre GCP, con el conector a Cloud Storage integrado

Cuadro 3: Explicación de los componentes de la arquitectura

Justificación técnica

- **GCP + Dataproc:** Dataproc integra Spark, Hadoop y HDFS con el conector a Cloud Storage ya configurado, sin tener que instalar ni administrar cada pieza por separado.
- **Un solo bucket:** cumple dos roles (staging del clúster y data lake del proyecto) organizados por carpetas, separarlo en dos buckets no aporta ningún beneficio funcional distinto para el tamaño de este proyecto.
- **Hardware del clúster:** `n4-standard-2` (2 vCPU, 8 GB RAM) $\times$ 3 nodos (1 master + 2 workers), con disco `hyperdisk-balanced` de 100 GB.
- **Batch, no streaming:** el dataset es un archivo histórico cerrado, no llegan registros nuevos en tiempo real.
- **Terraform:** toda la infraestructura se define como código y se provisiona o destruye de forma reproducible, sin pasos manuales, ver `terraform/README.md`.

3.3. Almacenamiento: bucket de Cloud Storage

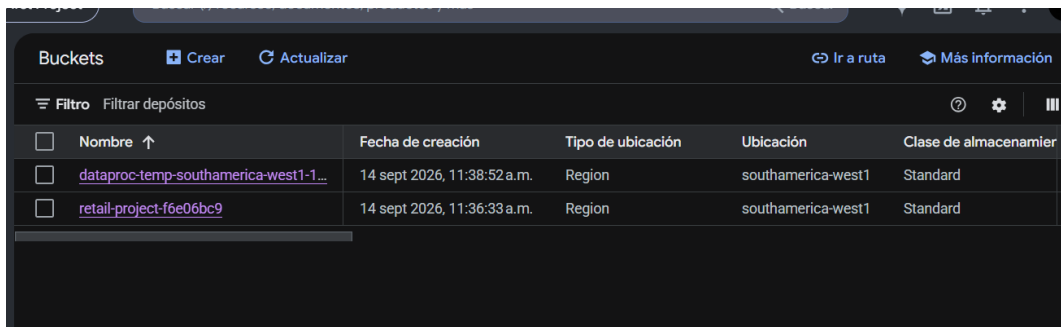
Se implementó un único bucket de Cloud Storage, organizado en tres carpetas que representan las etapas típicas de un data lake:

- *raw/*, el dataset original, sin modificar.
- *input/*, datos ya preparados para Hadoop MapReduce (columnas extraídas del CSV original).
- *output/*, resultados finales de los jobs de Hadoop MapReduce.

Este mismo bucket cumple además el rol de *staging bucket* del clúster Dataproc. Se decidió usar un único bucket con carpetas en vez de dos buckets separados porque, para el tamaño de este proyecto, separarlos no aporta ningún beneficio funcional adicional.

Evidencia

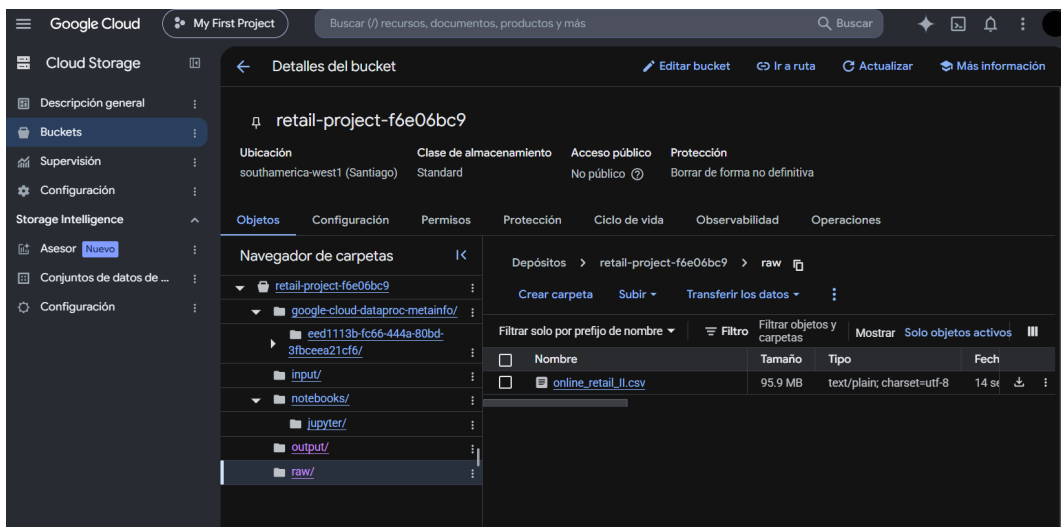
La Figura 2 muestra el bucket `retail-project-f6e06bc9` ya creado y visible en la consola de Google Cloud Storage del proyecto.



☐	Nombre ↑	Fecha de creación	Tipo de ubicación	Ubicación	Clase de almacenamier
☐	dataproc-temp-southamerica-west1-1...	14 sept 2026, 11:38:52 a.m.	Region	southamerica-west1	Standard
☐	retail-project-f6e06bc9	14 sept 2026, 11:36:33 a.m.	Region	southamerica-west1	Standard

Figura 2: Bucket del proyecto en la lista de Cloud Storage

Las Figuras 3, 4 y 5 muestran el contenido real de cada una de las tres carpetas.



Navegador de carpetas	
retail-project-f6e06bc9	
google-cloud-dataproc-metainfo/	
eed1113b-fc66-444a-80bd-3fbceea21cf6/	
input/	
notebooks/	
jupyter/	
output/	
raw/	

Depósitos > retail-project-f6e06bc9 > raw			
Crear carpeta Subir Transferir los datos			
Filtrar solo por prefijo de nombre Filtro Filtrar objetos y carpetas Mostrar Solo objetos activos			
☐	Nombre	Tamaño	Tipo
☐	online_retail_II.csv	95.9 MB	text/plain; charset=utf-8

Figura 3: Carpeta `raw/`: el dataset original (`online_retail_II.csv`, 95.9 MB)

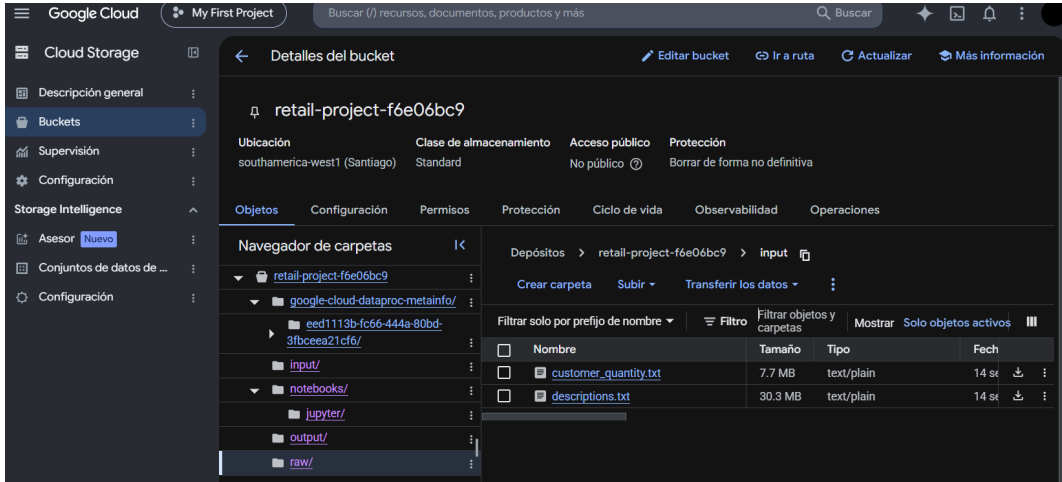


Figura 4: Carpeta `input/`: archivos preparados para Hadoop MapReduce

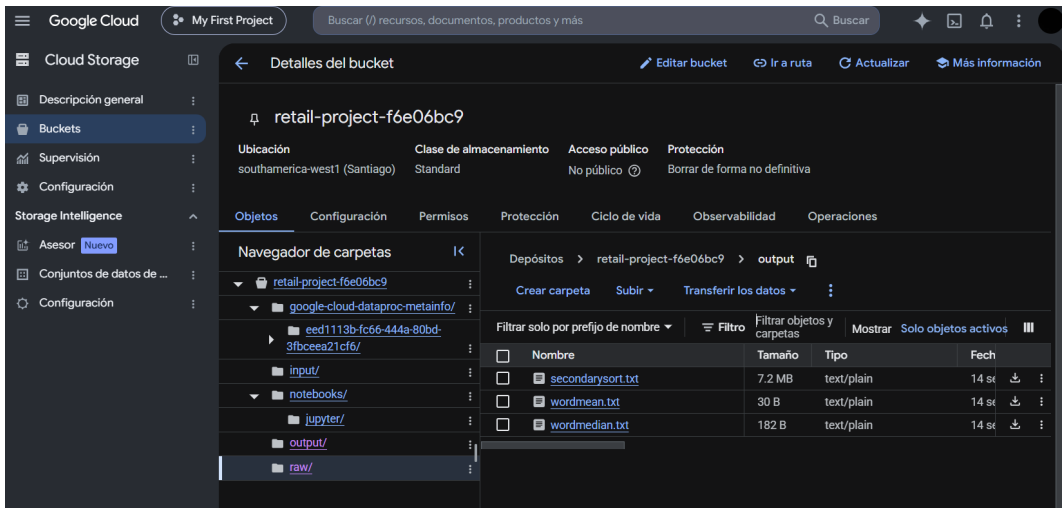


Figura 5: Carpeta `output/`: resultados de los tres programas de Hadoop MapReduce

Justificación del uso de almacenamiento en la nube

Se optó por Cloud Storage en vez de, por ejemplo, disco local del clúster, por tres razones concretas. Primero, Cloud Storage separa el cómputo del almacenamiento: los datos persisten en el bucket incluso si el clúster Dataproc se destruye, lo cual es clave porque el clúster se apaga y se vuelve a crear varias veces a lo largo del proyecto (y de hecho se apaga automáticamente todos los días como respaldo de costos, ver más abajo). Segundo, Dataproc integra de forma nativa un conector a Cloud Storage, por lo que tanto Spark como Hadoop MapReduce pueden leer y escribir directamente sobre `gs://` sin configuración adicional. Tercero, organizar el bucket en tres zonas (`raw/`, `input/`, `output/`) permite simular, con una sola herramienta, las etapas típicas de un data lake real: datos crudos, datos preparados, y resultados.

3.4. Clúster Dataproc

El clúster se aprovisionó en modo *Standard*: 1 nodo master + 2 nodos worker, todos `n4-standard-2` (2 vCPU, 8 GB RAM) con disco `hyperdisk-balanced` de 100 GB. Dataproc administra de forma nativa Spark, Hadoop y HDFS sobre este clúster, con el conector a Cloud Storage ya configurado.

Por seguridad, el clúster no tiene IP pública (`internal_ip_only = true`): el acceso es exclusivamente por SSH vía IAP tunneling o por el Component Gateway (JupyterLab, Spark History Server), nunca por una IP directa. Para que el clúster tenga salida a internet (necesaria para instalar dependencias de Python o clonar el repositorio) sin exponer una IP pública, se agregó un Cloud NAT (Cloud Router + Cloud NAT), que permite tráfico saliente sin aceptar ninguna conexión entrante.

Como respaldo de costos, se configuró un job de Cloud Scheduler que apaga el clúster automáticamente todos los días a una hora fija, por si el equipo olvida apagarlo manualmente.

Evidencia

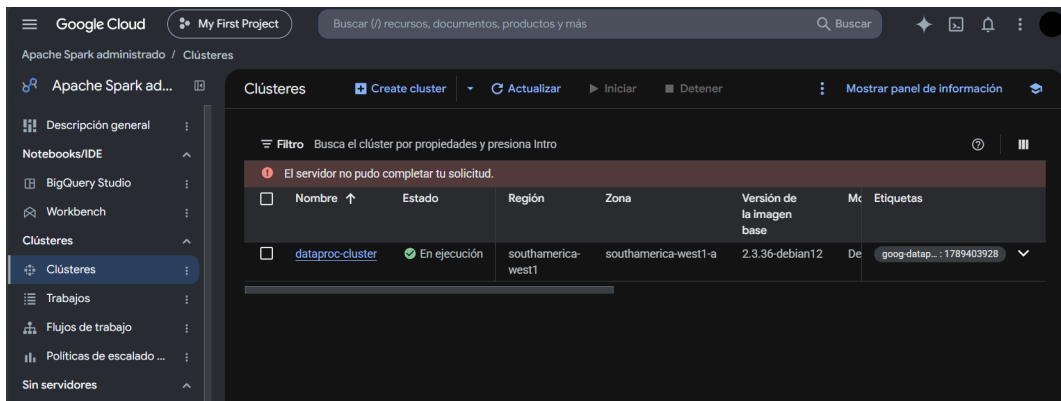


Figura 6: Clúster `dataproc-cluster` en estado *En ejecución*

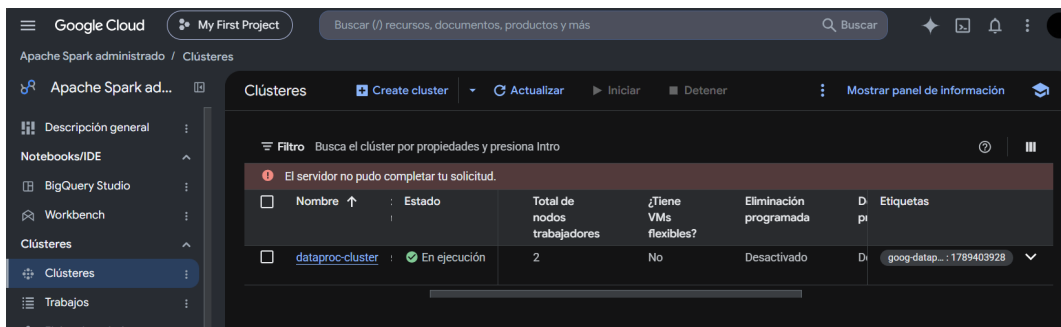


Figura 7: Detalle del clúster: 2 nodos trabajadores confirmados

3.5. Reproducibilidad con Terraform

Toda la infraestructura descrita en esta sección, bucket, clúster, service account de apagado automático y job de Cloud Scheduler, está definida en `terraform/main.tf`. Cualquier integrante del equipo puede levantar una copia idéntica de esta infraestructura en su propio proyecto de GCP corriendo `terraform apply`, y destruirla por completo con `terraform destroy`, sin dejar recursos huérfanos ni pasos manuales pendientes.

4. Procesamiento distribuido

4.1. Metodología

Se definió un único conjunto de 12 consultas de negocio, y se implementó exactamente el mismo conjunto en los cuatro frameworks pedidos por el enunciado: Polars, Dask, Modin y Apache Spark. La razón de usar el mismo set de consultas en los cuatro, en vez de un set distinto por framework, es que el objetivo de

esta sección es comparar cómo cada herramienta resuelve el mismo problema, no evaluar cuatro análisis de negocio independientes. Cada framework lee el CSV original desde cero y corre el pipeline completo de forma independiente (no reutiliza resultados intermedios de otro framework).

#	Categoría(s)	Consulta
1	Limpieza	Eliminar filas con <code>Description</code> nula o <code>Price</code> ≤ 0
2	Duplicados	Detectar y eliminar filas exactamente duplicadas
3	Transformación	Crear <code>TotalPrice</code> , parsear fecha, derivar <code>Year/Month</code> , <code>IsCancelled</code>
4	Filtrado	Quedarse solo con ventas válidas (cantidad y precio positivos, no canceladas)
5	Agrupación + Agregación	Top 10 productos por revenue total
6	Agrupación + Agregación	Top 10 productos por unidades vendidas
7	Agrupación + Ordenamiento	Top 10 clientes por gasto total
8	Agrupación + Ordenamiento	Revenue y ticket promedio por país (top 10)
9	Métrica temporal	Revenue mensual y variación % mes a mes
10	Métrica + Filtrado	Tasa de cancelación por país (top 10)
11	Agrupación + Métrica	Segmentación de clientes en 3 tiers de gasto
12	Métrica	Ticket promedio (AOV) global y su evolución mensual

Cuadro 4: Las 12 consultas implementadas en cada uno de los 4 frameworks

Las consultas 1 a 4 preparan el dataset (limpieza, deduplicación, transformación, filtrado), las consultas 5 a 12 son las preguntas de negocio propiamente dichas, y se calculan sobre el subconjunto ya limpio de ventas válidas (salvo la consulta 10, que necesita ver también las transacciones canceladas para calcular una tasa).

4.2. Polars

Polars se implementó usando su API de expresiones (`with_columns`, `filter`, `group_by().agg()`), en modo eager. A diferencia de los otros tres frameworks, Polars no tiene ningún modo distribuido multi-máquina en su versión de código abierto: es una librería de una sola máquina por diseño, que paraleliza internamente entre los núcleos del procesador mediante su motor en Rust. Esto no es una limitación del proyecto: para consultas de este tamaño (1 millón de filas), Polars resultó ser, en la práctica, el más simple de implementar y uno de los más rápidos localmente.

Se usó `qcut` para la segmentación de clientes (consulta 11) y `pct_change` para la variación mensual (consulta 9), ambos disponibles de forma nativa en la API de Polars.

Como referencia, así se ve la consulta 5 (top 10 productos por revenue) en Polars:

```
top10 = (
    df.group_by("Description")
      .agg(pl.col("TotalPrice").sum())
      .sort("TotalPrice", descending=True)
      .head(10)
)
```

4.3. Apache Spark

Spark se implementó con la API de `DataFrame` (`groupBy().agg()`, funciones del módulo `pyspark.sql.functions`), usando una función de ventana (`Window + lag`) para la variación mensual y `approxQuantile` para la segmentación de clientes, ya que Spark no tiene un equivalente directo a `qcut`.

Spark es, de los cuatro, el único que se ejecutó también sobre el clúster real de Dataproc en modo genuinamente distribuido (ver Sección 4.6): al correrlo con `spark-submit` sin forzar un `master` local, el job se somete a YARN y se reparte entre el nodo master y los dos workers.

Como referencia, así se ve la consulta 5 (top 10 productos por revenue) en Spark:

```

top10 = (
    df.groupby("Description")
      .agg(F.sum("TotalPrice").alias("TotalPrice"))
      .orderBy(F.desc("TotalPrice"))
      .limit(10)
)

```

4.4. Modin

Modin se implementó usando `modin.pandas`, que expone una API prácticamente idéntica a pandas, de hecho, la implementación es casi un calco de la de Dask, quitando las llamadas a `.compute()`. Por debajo, Modin distribuye el trabajo usando Ray como motor de ejecución. Al importar `modin.pandas` sin conectarse explícitamente a un clúster Ray existente, Ray arranca automáticamente una instancia local sobre la máquina donde corre el script.

Como referencia, así se ve la consulta 5 (top 10 productos por revenue) en Modin, prácticamente idéntica a pandas:

```

resultado = df.groupby("Description")["TotalPrice"].sum()
top10 = resultado.nlargest(10)

```

4.5. Dask

Dask se implementó con `dask.dataframe`, cuya API está deliberadamente inspirada en pandas. Un detalle técnico relevante: al leer el CSV con un `blocksize` demasiado grande, Dask particionaba el archivo en una sola partición, lo que anulaba cualquier paralelismo real, se ajustó el `blocksize` para obtener varias particiones reales. También se agregó `.persist()` después de las etapas de limpieza y filtrado, para que el pipeline pesado no se recomputara desde cero en cada una de las ocho consultas de negocio restantes (Dask es perezoso por defecto).

Como referencia, así se ve la consulta 5 (top 10 productos por revenue) en Dask, con el `.compute()` explícito que dispara la ejecución:

```

resultado = df.groupby("Description")["TotalPrice"].sum().compute()
top10 = resultado.nlargest(10)

```

4.6. Resultados y comparación

Las cuatro implementaciones, corriendo de forma completamente independiente, producen resultados consistentes entre sí en las 12 consultas: mismo top de productos y clientes, mismo ticket promedio, mismas tasas de cancelación. La única excepción es la consulta 11 (segmentación de clientes), donde Polars ubica a un cliente en un tier distinto al de los otros tres frameworks, el detalle de esa diferencia puntual se explica en la Sección 5. Fuera de ese caso, las cuatro coinciden exactamente, lo que confirma que resuelven correctamente el mismo problema, solo con sintaxis y motor de ejecución distintos.

La Tabla 5 muestra que las 12 consultas de la Tabla 4 fueron implementadas y ejecutadas con éxito en los 4 frameworks, con resultado numérico coincidente en 11 de las 12 (la excepción es la consulta 11, ver Sección 5).

#	Consulta	Polars	Dask	Modin	Spark
1	Limpieza	✓	✓	✓	✓
2	Duplicados	✓	✓	✓	✓
3	Transformación	✓	✓	✓	✓
4	Filtrado	✓	✓	✓	✓
5	Top 10 productos por revenue	✓	✓	✓	✓
6	Top 10 productos por unidades	✓	✓	✓	✓
7	Top 10 clientes por gasto	✓	✓	✓	✓
8	Revenue y ticket por país	✓	✓	✓	✓
9	Revenue mensual y variación	✓	✓	✓	✓
10	Tasa de cancelación por país	✓	✓	✓	✓
11	Segmentación de clientes	✓*	✓	✓	✓
12	Ticket promedio (AOV)	✓	✓	✓	✓

Cuadro 5: Cobertura de las 12 consultas por framework, implementadas y ejecutadas con éxito en los 4, con resultado numérico coincidente en 11 de las 12 filas

*Polars ubica a un cliente en un tier de gasto distinto al de los otros tres en la consulta 11, ver el detalle en la Sección 5.

Corrida sobre el clúster real de Dataproc

Además de validarse en local, los cuatro frameworks se corrieron también sobre el clúster real de Dataproc descrito en la Sección 3, leyendo el dataset directamente desde el bucket. Los cuatro reprodujeron exactamente los mismos resultados que en local. El output completo de cada corrida está disponible en `docs/evidencia/` (un archivo `-cluster-output.txt` por framework).

Vale la pena aclarar que, de los cuatro, solo Spark aprovecha genuinamente los tres nodos del clúster: Dataproc ya trae YARN configurado de fábrica, así que basta con no forzar un `master` local en el código. Dask y Modin, al no conectarse explícitamente a un scheduler distribuido (Dask) o a un clúster Ray (Modin) montado entre los nodos, corren en el nodo master usando sus propios núcleos, siguen ejecutándose sobre infraestructura de Dataproc, pero sin repartirse entre los tres nodos. Montar un clúster Dask o Ray real entre los tres nodos es posible, pero es trabajo de infraestructura adicional que no aporta valor proporcional al alcance de este proyecto.

4.7. Comparación de rendimiento

Para comparar objetivamente los cuatro frameworks bajo las mismas condiciones, se corrió el pipeline completo (carga del CSV más las 12 consultas) una vez por framework, en la misma máquina y en un entorno virtual aislado por framework, cada uno con exactamente las versiones fijadas en su `requirements.txt`. Se midió el tiempo de pared de cada etapa con `time.perf_counter` y la memoria RSS pico (proceso principal más todos sus procesos hijos, muestreada cada 100 ms) con la librería `psutil`. El script usado es `processing/run_benchmark.py` y la salida cruda de cada corrida está en `docs/evidencia/benchmark/`.

Esta medición es intencionalmente distinta de la corrida en el clúster de Dataproc: aquí las 4 corren en modo local de una sola máquina (Intel Core i7-13620H, 10 núcleos / 16 hilos, 16 GB RAM, Windows 11), precisamente para que la comparación sea justa entre sí, ya que en el clúster solo Spark se distribuye genuinamente entre nodos (ver más arriba).

Framework	Tiempo total (s)	Memoria pico (MB)
Polars	0.78	915
Dask	10.35	731
Modin	16.45	5,937
Spark (local)	134.44	1,511

Cuadro 6: Tiempo total y memoria pico del pipeline completo (carga + 12 consultas), corrida local

Polars resultó, por lejos, el más rápido y uno de los más livianos: al ser una librería de una sola máquina optimizada en Rust, sin overhead de coordinación entre procesos, no tiene que pagar ningún costo de inicialización. Dask queda en un punto intermedio. Modin usa más memoria que los demás porque Ray reserva por adelantado un object store compartido para su motor de ejecución, un costo fijo que se paga incluso para un dataset de este tamaño. Spark en modo local es, de los cuatro, el más lento: paga el costo de levantar una JVM completa y de armar un plan de ejecución distribuido pensado para clústers de muchos nodos, un costo que no se amortiza corriendo en una sola máquina con un dataset de ~ 1 millón de filas.

La Tabla 7 desglosa el tiempo por etapa, para ver en qué parte del pipeline se concentra el costo de cada framework.

Etapas	Polars	Dask	Modin	Spark
Inicialización de motor	—	—	—	4.75
Carga del CSV	0.05	0.01	10.43	0.63
1. Limpieza	0.02	1.86	0.81	2.42
2. Duplicados	0.12	3.14	0.99	3.42
3. Transformación	0.25	4.12	0.37	3.12
4. Filtrado	0.01	0.63	0.39	6.41
5. Top productos (revenue)	0.06	0.03	0.21	15.05
6. Top productos (unidades)	0.04	0.03	0.19	12.59
7. Top clientes	0.02	0.04	0.28	12.19
8. Revenue por país	0.07	0.16	0.58	13.32
9. Revenue mensual	0.02	0.03	0.46	2.38
10. Tasa de cancelación	0.04	0.05	0.27	2.75
11. Segmentación de clientes	0.03	0.03	0.33	25.58
12. Ticket promedio (AOV)	0.05	0.21	1.14	28.68

Cuadro 7: Tiempo por etapa del pipeline, en segundos (corrida local, un framework a la vez)

Algunos patrones se explican directamente por cómo trabaja cada motor. En Modin, la carga del CSV concentra la mayor parte del tiempo total porque ahí es cuando Ray arranca su instancia local, el resto de las consultas son rápidas una vez que el motor ya está en pie. En Dask, en cambio, el costo se concentra en las etapas 1 a 3: como es perezoso, esas etapas son las primeras en forzar una lectura real del CSV completo, las consultas de negocio posteriores reutilizan los datos ya materializados con `.persist()` y por eso son casi instantáneas. En Spark local, las consultas 5, 6, 7, 8, 11 y 12 son las más caras porque cada una dispara una acción (`show()` o `first()`) que fuerza a la JVM a planificar y ejecutar desde cero sobre el DataFrame cacheado, el overhead de planificación de Spark pesa más que el trabajo real cuando el dataset cabe cómodo en memoria de una sola máquina.

El mismo benchmark, corrido en el clúster Dataproc

El benchmark local de arriba compara los 4 frameworks bajo las mismas condiciones, pero deliberadamente deja fuera el escenario donde Spark debería tener ventaja: repartido entre nodos reales. Para no dejar esa comparación solo como advertencia en el texto, se corrió el mismo `processing/run_benchmark.py` directamente en el nodo master del clúster (`n4-standard-2`, 2 vCPU, 8 GB RAM) para Polars, Dask y Modin, y para Spark se envió como job real de YARN (`gcloud dataproc jobs submit pyspark`), repartido entre el master y los 2 workers, en vez de correrlo como proceso local. La salida cruda de las 4 corridas está en `docs/evidencia/benchmark/*-cluster.json`.

Framework	Tiempo total (s)	Dónde corrió
Polars	1.76	nodo master (proceso local)
Dask	11.25	nodo master (proceso local)
Modin	12.40	nodo master (proceso local)
Spark	170.65	job de YARN, 3 nodos

Cuadro 8: Tiempo total del mismo pipeline, corrido en el clúster de Dataproc de prueba

El resultado es honesto pero no el esperado: Spark corriendo distribuido en YARN (170.65 s) resultó más lento que corriendo en modo local en la laptop (134.44 s), no más rápido. Esto no contradice lo dicho en la Sección 4.6 sobre que Spark es el único que reparte trabajo entre nodos, solo confirma algo distinto: para un dataset de ~ 1 millón de filas, en un clúster de prueba con máquinas modestas (**n4-standard-2**, 2 vCPU cada una), el overhead de coordinar tareas y mover datos entre 3 nodos por red pesa más que el beneficio de paralelizar el trabajo. Ese overhead se amortiza con datasets mucho más grandes o clústers con más núcleos por nodo, no con este volumen de datos ni este hardware. Polars, Dask y Modin, al no distribuirse entre nodos ni en el clúster ni en local, simplemente corren más lento en el master del clúster que en la laptop porque el master tiene menos núcleos y memoria (2 vCPU / 8 GB frente a 10 núcleos / 16 GB).

5. Operaciones

Esta sección detalla, con resultados concretos, las operaciones ejecutadas sobre los datos en las 12 consultas descritas en la Sección 4. Los valores presentados aquí coinciden en los cuatro frameworks, con la única excepción de la segmentación de clientes (consulta 11), detallada más adelante en esta sección.

5.1. Operaciones tipo CRUD sobre el dataset

Aunque el dataset es un archivo estático (no una base de datos transaccional), el pipeline aplica sobre él las cuatro operaciones clásicas de CRUD en un sentido batch:

- **Create:** se crean columnas derivadas que no existen en el CSV original: **TotalPrice** (**Quantity** $\times$ **Price**), **Year**, **Month** e **IsCancelled** (booleano, a partir del prefijo **C** en **Invoice**).
- **Read:** lectura del CSV completo con un esquema de tipos explícito por columna, para evitar inferencias erróneas de tipo.
- **Update:** **InvoiceDate** se convierte de texto a tipo fecha-hora real, habilitando operaciones temporales (**Year**, **Month**) que no son posibles sobre una columna de texto.
- **Delete:** eliminación de filas duplicadas exactas y de filas que no representan ventas reales (**Description** nula, **Price** ≤ 0 , o cancelaciones).

5.2. Validación y limpieza

Etapa	Filas antes	Filas después
Limpieza (Description nula o Price ≤ 0)	1,067,371	1,061,164
Eliminación de duplicados exactos	1,061,164	1,027,017
Filtrado de ventas válidas	1,027,017	1,007,913

Cuadro 9: Efecto de cada etapa de validación sobre el número de filas

En total, de las 1,067,371 filas originales, 1,007,913 (94.4 %) corresponden a ventas efectivamente válidas. El 5.6 % restante corresponde a registros basura (precio nulo o cero), duplicados exactos, y transacciones canceladas.

5.3. Agrupaciones y agregaciones

Top 10 productos por revenue

Producto	Revenue (£)
Manual	339,241.29
REGENCY CAKESTAND 3 TIER	330,590.32
DOTCOM POSTAGE	309,854.11
WHITE HANGING HEART T-LIGHT HOLDER	260,990.22
PAPER CRAFT, LITTLE BIRDIE	168,469.60
PARTY BUNTING	148,318.28
JUMBO BAG RED RETROSPOT	148,073.47
ASSORTED COLOUR BIRD ORNAMENT	129,324.49
POSTAGE	125,682.42
PAPER CHAIN KIT 50'S CHRISTMAS	117,760.29

Cuadro 10: Top 10 productos por revenue total

Es interesante notar que dos de las diez primeras filas (**Manual** y **DOTCOM POSTAGE/POSTAGE**) no son productos reales, sino líneas contables internas del negocio (ajustes manuales y cargos de envío). Esto es, en sí mismo, un hallazgo de calidad de datos: el dataset original no separa productos de conceptos administrativos, algo a tener en cuenta si se quisiera refinar aún más la limpieza.

Top 10 clientes por gasto total

Customer ID	Gasto total (£)
18102.0	580,987.04
14646.0	528,602.52
14156.0	313,437.62
14911.0	291,420.81
17450.0	244,784.25
13694.0	195,640.69
17511.0	172,132.87
16446.0	168,472.50
16684.0	147,142.77
12415.0	144,458.37

Cuadro 11: Top 10 clientes por gasto total

Revenue y ticket promedio por país

País	Revenue (£)	Facturas	Ticket promedio (£)
United Kingdom	17,410,196.12	36,535	476.53
EIRE	658,767.31	626	1,052.34
Netherlands	554,038.09	228	2,429.99
Germany	425,019.71	789	538.68
France	350,456.09	622	563.43
Australia	169,283.46	95	1,781.93
Spain	108,332.49	154	703.46
Switzerland	100,685.59	93	1,082.64
Sweden	91,869.82	105	874.95
Denmark	68,580.69	43	1,594.90

Cuadro 12: Top 10 países por revenue, con ticket promedio

El Reino Unido concentra la gran mayoría de las transacciones (36,535 facturas) pero con el ticket promedio más bajo del top 10, en contraste, Países Bajos tiene apenas 228 facturas pero el ticket promedio más alto (£2,429.99), lo que sugiere clientes mayoristas con pedidos grandes y poco frecuentes.

5.4. Métricas de negocio

Tasa de cancelación por país

País	Total	Canceladas	Tasa
USA	535	126	23.55 %
Japan	565	97	17.17 %
Czech Republic	30	5	16.67 %
Korea	63	10	15.87 %
United Arab Emirates	500	33	6.60 %
Nigeria	32	2	6.25 %
Channel Islands	1,646	95	5.77 %
Malta	299	17	5.69 %
Australia	1,887	98	5.19 %
Germany	17,331	899	5.19 %

Cuadro 13: Top 10 países con mayor tasa de cancelación (mínimo 30 transacciones)

Segmentación de clientes por gasto

Se segmentaron los 5,942 clientes únicos en tres tiers de gasto total (bajo, medio, alto) usando terciles, resultando en una distribución equilibrada de alrededor de 1,960 clientes por tier. Esta es la única consulta donde los cuatro frameworks no coinciden exactamente. Dask, Modin y Spark ubican a 1,959 clientes en el tier Bajo y a 1,960 en el tier Medio, mientras que Polars ubica a 1,960 en Bajo y a 1,959 en Medio. La diferencia es un solo cliente, ubicado justo en el límite entre ambos tiers, y se debe a que cada framework calcula los puntos de corte del tercil con un método distinto: Dask aplica `pandas.qcut` sobre el resultado ya agregado, Modin usa el método equivalente de su propia API, Spark calcula los cuantiles de forma exacta con `approxQuantile (relativeError=0)`, y Polars usa su propio `qcut`. Ninguno de los cuatro está mal. Simplemente no todos definen igual el punto donde termina un tercil y empieza el siguiente.

Ticket promedio (AOV) global y su evolución

El ticket promedio global (AOV) sobre las ventas válidas es de **£510.92**. Al observar su evolución mensual junto con el revenue mensual (consulta 9), se aprecia un patrón estacional claro: los meses de noviembre de cada año (previos a la temporada navideña) concentran los picos más altos de revenue, por ejemplo, noviembre de 2011 alcanzó £1,503,867 (+30.6 % respecto al mes anterior). Diciembre de 2011 presenta un total mensual menor, pero los datos solo cubren hasta el día 9, por ello, no es directamente comparable con noviembre ni permite atribuir la caída a los plazos de envío navideños.

6. Hadoop MapReduce

Como parte del proyecto, el grupo ejecutó tres programas incluidos en `hadoop-mapreduce-examples.jar` sobre el clúster de Dataproc, usando HDFS como sistema de archivos distribuido, tal como exige el enunciado. Se eligieron `wordmean`, `wordmedian` y `secondarysort`, cualquier programa del jar está permitido salvo `wordcount` y `grep`, por tener sintaxis simple (`<input><output>`, sin argumentos ambiguos) y por poder aplicarse directamente sobre columnas reales del dataset.

6.1. Preparación de los inputs

A partir del CSV original se extrajeron dos archivos de texto plano:

- `descriptions.txt`, una descripción de producto (columna `Description`) por línea.
- `customer_quantity.txt`, pares de valores `CustomerID` y `Quantity` separados por un espacio, uno por línea.

Ambos se subieron a `gs://<bucket>/input/` y de ahí se copiaron a HDFS (`/retail/`) antes de correr los jobs.

6.2. Los tres programas

Programa	Objetivo	Input
<code>wordmean</code>	Largo promedio de palabra	<code>descriptions.txt</code>
<code>wordmedian</code>	Mediana del largo de palabra	<code>descriptions.txt</code>
<code>secondarysort</code>	Agrupar por clave y ordenar por una segunda	<code>customer_quantity.txt</code>

Cuadro 14: Programas de `hadoop-mapreduce-examples.jar` utilizados

`wordmean` y `wordmedian` dan una idea de la consistencia del texto en el catálogo de productos: en un catálogo de e-commerce con miles de productos distintos, saber si las descripciones son consistentemente cortas o largas es relevante para tareas posteriores de procesamiento de texto (búsqueda, categorización). `secondarysort` agrupa todas las compras de cada cliente (`Customer ID`) y las ordena internamente por cantidad (`Quantity`), lo que permite ver, por cliente, si sus compras son consistentemente pequeñas o si tiene algunos pedidos grandes puntuales. Este último programa demuestra además el patrón de *secondary sort* de MapReduce: agrupar por una clave mientras se ordena por una segunda dentro de cada grupo, algo que requiere una lógica de partición/orden compuesta y no es trivial de lograr con un `reduce` simple.

6.3. Comando ejecutado

Cada job se envió mediante `gcloud dataproc jobs submit hadoop`, la forma nativa de correr jobs en Dataproc, que hace *polling* del estado del job vía la API sin depender de una conexión SSH persistente al clúster. El comando exacto ejecutado para cada uno de los tres programas fue:

```
gcloud dataproc jobs submit hadoop \
  --cluster=dataproc-cluster --region=southamerica-west1 \
  --jar=file:///usr/lib/hadoop-mapreduce/hadoop-mapreduce-examples.jar \
  -- <programa> <input_hdfs> <output_hdfs>
```

donde los tres parámetros toman los siguientes valores según el programa que se esté ejecutando:

Parámetro	Valor
<code><programa></code>	<code>wordmean</code> , <code>wordmedian</code> o <code>secondarysort</code>
<code><input_hdfs></code>	<code>/retail/descriptions.txt</code> o <code>/retail/customer_quantity.txt</code> , según corresponda
<code><output_hdfs></code>	<code>/retail/<programa></code>

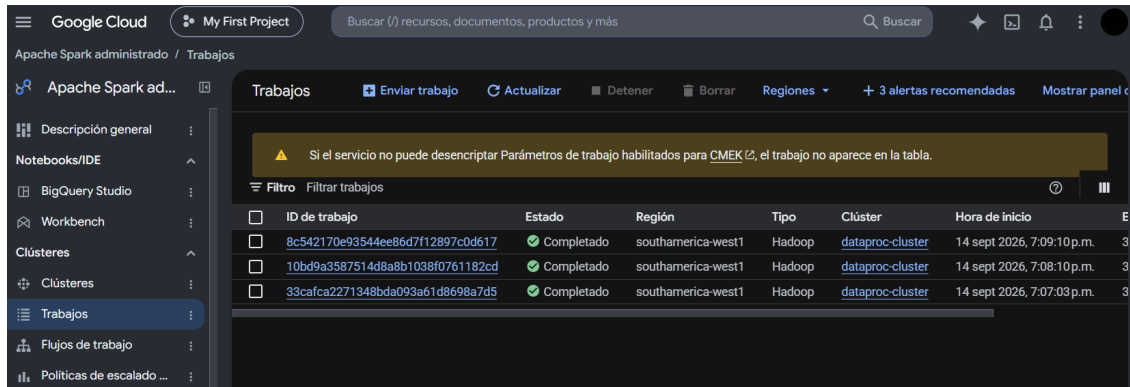
Cuadro 15: Parámetros del comando de envío de jobs Hadoop

El resultado de cada job (las particiones `part-r-00000`, `part-r-00001`, ... generadas por los *reducers*) queda almacenado en HDFS, dentro del clúster, tal como exige el enunciado. Recién después, para poder explorarlo y conservarlo más allá de la vida del clúster, se juntan esas particiones en un solo archivo y se copian a Cloud Storage:

```
hadoop fs -getmerge /retail/<programa> /tmp/<programa>.txt
gsutil cp /tmp/<programa>.txt gs://<bucket>/output/<programa>.txt
```

6.4. Evidencia y resultados

La Figura 8 muestra los tres jobs completados exitosamente sobre el clúster.



The screenshot shows the Google Cloud console interface for 'My First Project'. The left sidebar lists various services, with 'Trabajos' (Jobs) selected. The main panel displays a table of jobs. A yellow warning banner at the top states: 'Si el servicio no puede descriptar Parámetros de trabajo habilitados para CMEK, el trabajo no aparece en la tabla.' The table has columns for 'ID de trabajo', 'Estado', 'Región', 'Tipo', 'Clúster', 'Hora de inicio', and 'E'. Three jobs are listed, all with a status of 'Completado' (Completed).

ID de trabajo	Estado	Región	Tipo	Clúster	Hora de inicio	E
8c542170e93544ee86d7f12897c0d617	Completado	southamerica-west1	Hadoop	dataproc-cluster	14 sept 2026, 7:09:10 p.m.	3
10bd9a3587514d8a8b1038f0761182cd	Completado	southamerica-west1	Hadoop	dataproc-cluster	14 sept 2026, 7:08:10 p.m.	3
33cafa2271348bda093a61d8698a7d5	Completado	southamerica-west1	Hadoop	dataproc-cluster	14 sept 2026, 7:07:03 p.m.	3

Figura 8: Los tres jobs de Hadoop MapReduce en estado *Completado*

`wordmean.txt`: el programa no imprime la media directamente, sino los contadores con los que se calcula:

```
count    4664518
length   24485085
```

$\text{Media} = \text{length} / \text{count} \approx 5.25$ caracteres por palabra.

`wordmedian.txt`: tabla de frecuencia que relaciona el largo de palabra con la cantidad de palabras de ese largo. La mediana se obtiene acumulando estos conteos, ordenados por largo, hasta llegar al 50 % del total de palabras.

`secondarysort.txt`: compras agrupadas por `Customer ID`, ordenadas ascendentemente por `Quantity` dentro de cada grupo (aproximadamente 7 MB de resultado). Por ejemplo, para el cliente 12348:

```
12348    1
12348    1
12348    1
12348    6
12348   12
...
```

Estos tres resultados fueron reproducidos de forma independiente sobre un clúster Dataproc real, obteniendo valores idénticos a los documentados originalmente por el equipo (ver `hadoop-mapreduce/README.md`), lo que confirma que el pipeline es completamente reproducible.

7. Anexo: Repositorio del proyecto

El código completo del proyecto, incluyendo las cuatro implementaciones de procesamiento (Polars, Dask, Modin, Spark), la infraestructura Terraform, los notebooks de exploración, los programas de Hadoop MapReduce, la evidencia recolectada y este mismo informe en LaTeX, está publicado en el siguiente repositorio de GitHub:

<https://github.com/KarTaGo124/Big-Data-Project-I>

- `processing/`, implementaciones de las 12 consultas en Polars, Dask, Modin y Spark.
- `terraform/`, infraestructura de GCP como código (bucket, clúster Dataproc, Cloud NAT, Cloud Scheduler).

- `hadoop-mapreduce/`, resultados y documentación de los tres programas de Hadoop.
- `docs/evidencia/`, capturas y logs de las corridas sobre el clúster real.
- `docs/informe/`, código fuente de este informe.